

Session With **Sumit**



**Sumit
Shukla**

WHAT ARE LLMs?

LARGE LANGUAGE MODELS

Hey, Do you also feel tired from work?

As a data scientist and content creator, I'm always engaged in the hectic schedule with tons of tasks. Today, I was wondering if I could make my life easier by assigning my tasks to a machine and the machine, in turn, completes my work. Wouldn't it be great if humans could talk to robots, and robots could understand our language and make our work easier? Just imagine yourself communicating with computers in a way that feels just like talking to a human. So I decided to research and found a model that can help me!

Can you guess?

It's called Large Language Model (LLM).

In this article, I will discuss LLMs and uncover how they work, their benefits, and the incredible impact they have on our world.



So, what is a Large Language Model?

A Large Language Model is a special kind of artificial intelligence (AI) technology that is trained to understand and generate human-like text. It's like a virtual brain that can process vast amounts of information, learn from it, and then use that knowledge to answer questions, have conversations, and even write stories, articles, or poems!

But there are language models and large language models (LLMs), both of them learn the patterns and understand the hidden meaning of the sentences but they are different.

Language models follow probabilistic-based approach while LLMs are trained on huge data with billions of parameters.



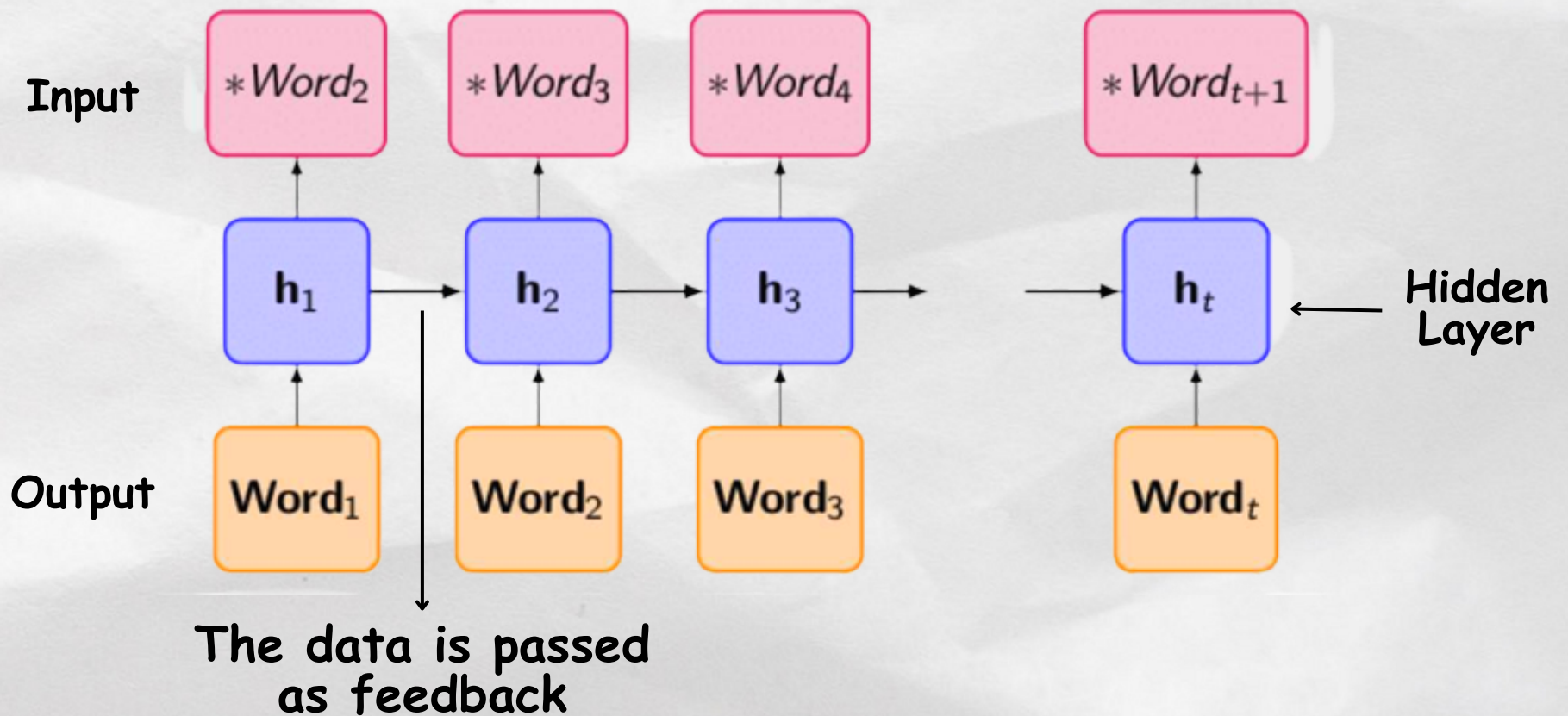
Sounds
interesting, right?
Let me make it
more simpler!

A language model is like a computer program that tries to understand how words fit together. It looks at a lot of text and learns the patterns and probabilities of different word sequences. There are different types of language models, some simple and some more advanced.

Let's consider
one such model
called RNN.



Below, is the RNN(Recurrent Neural Network) model, a type of Language model that takes input as a word and then a hidden layer processes it. Here, the hidden layer act as a memory which stores information about the previous inputs (or words). The output of the hidden layer is used as the feedback for the next word in sequence.



A large language model is an advanced version of the language model. It has a lot of parameters that help understand and generate text. They are trained on huge amounts of data like books, magazines, web pages, etc.

You can think of these parameters as the features that are extracted while training the model.



Hey I know you
are wondering
how they really
work!

These models are really big and can understand complex patterns in language. Internally, they also use neural networks that are trained on large amounts of data.

When they generate text, it often looks just like a person would write.

So, a large language model is like a super brain that can understand and create text that sounds just like humans.



Now, you must be wondering where are they used?

They are used to solve multiple NLP problems like:

1. **Sentiment Analysis**: Figuring out if a piece of text (like a tweet or a review) is positive, negative, or neutral.
2. **Named Entity Recognition**: Identifying and categorizing specific types of information in text, like names of people, places, organizations, or dates.
3. **Machine Translation**: Automatically translating text from one language to another, like converting English sentences to French.
4. **Text Classification**: Grouping text into different categories based on its content, such as classifying emails as spam or legitimate.

5. **Question Answering:** Understanding a question and providing a relevant answer based on a given context or knowledge base.

6. **Text Summarization:** Condensing a long piece of text into a shorter summary, capturing the main points and important information.

7. **Text Generation:** Generating new text based on given prompts or patterns, like creating a poem or a story.

8. **Named Entity Disambiguation:** Resolving ambiguity when a named entity, like "Apple," can refer to either a fruit or a company, based on the context.

9. **Language Modeling:** Predicting the next word in a sentence or generating coherent text based on previous words.

10. **Speech Recognition:** Converting spoken language into written text, like transcribing audio recordings or voice commands.

Now, Let's look at the examples of LLMs that can make our lives easier



Here are a few examples of Large Language Models (LLMs) and their usage:

1. **GPT-3 (Generative Pre-trained Transformer):** One of the most famous LLMs developed by OpenAI, it can perform variety of tasks such as language translation, content generation, question answering, and even coding assistance. It has been used to create conversational agents, chatbots, and virtual assistants.



As a content creator, I am a huge fan of ChatGPT as it has helped me in doing many tasks like - writing content, language translation, etc. Did you try GPT to do any of your task?

2. **BERT (Bidirectional Encoder Representations from Transformers)**: BERT is another popular LLM developed by Google. It excels in understanding the context and meaning of words in a sentence. It has been used for tasks like sentiment analysis, named entity recognition, text classification, and question answering.

3. **T5 (Text-to-Text Transfer Transformer)**: It is another example of a large language model. It is a transformer-based model developed by Google Research.

T5 can be fine-tuned to perform different tasks like text classification, text generation, translation, summarization, and more.[f]



But why are there so many LLMs?
What's different about each of them?

They all have different model architectures, different pretraining objectives and fine-tune flexibility.
(which will be explained in the next article)



Now the most important question arises
- How do these LLMs work?

Before understanding the working and architecture, it is important to know certain terminologies.

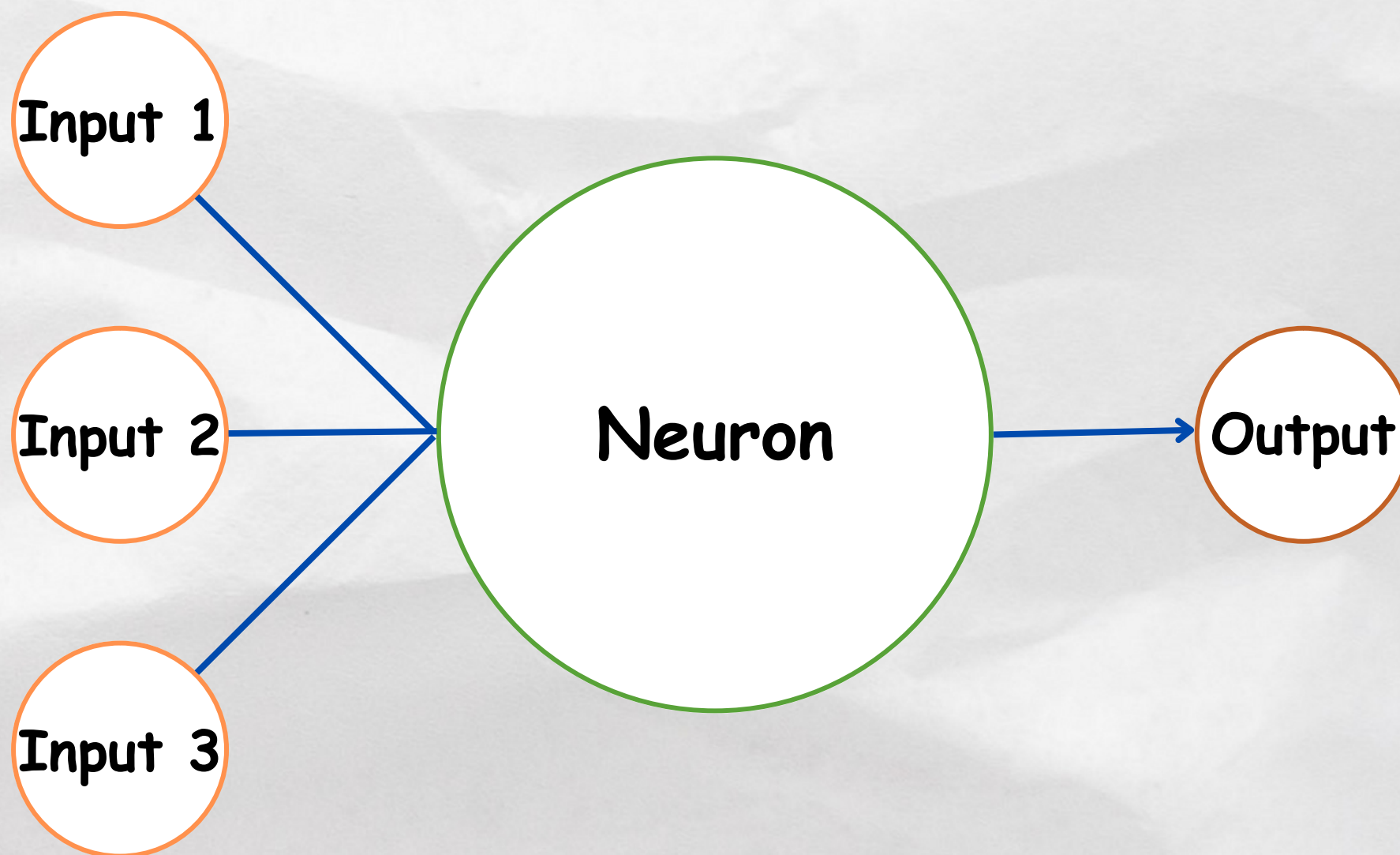
First, we will look how training is done:

LLMs are trained using a large amount of text data from books, articles, websites, etc. They learn from this data to understand the patterns and structures of language. More is the data, better is their capability of capturing textual patterns.

Neural Networks:

LLMs use neural networks which are made up of interconnected nodes or "neurons." Each neuron takes an input and produces an output.

You can think of a neuron as an algorithm which takes input, some mathematical function is performed in which the neuron processes input and gives output.



Word Embeddings:

In machine learning problems, we usually normalize or standardize our data before modelling. Word Embeddings are a way to standardize and normalize the textual data so that the meaning also gets captured.

Whenever we input a textual data, it should be in form of numbers rather than text as the computer can understand only numbers. So, it is required to convert textual data to number. This can be achieved with the help of word embeddings.

In the case of language models, words are represented as vectors called "word embeddings." Word embeddings capture the meaning and relationships between words.

For example, we need to pass the word "cat" in the network to be processed. Now, the computer cannot understand this it is important to be converted to some numbers. So, for we will perform methods like one hot encoding, bag of words etc. and then it will be converted to vector. So after processing, cat is represented as $[0,0,0,1,0,0]$



Context and Probability:

LLMs analyze text by considering the context of the words. They look at a sequence of words and predict what comes next based on the previous words.

This is done by assigning probabilities to different possible next words.

For example, given the sentence "I want to eat an ice...", the LLM might predict "cream" as the most likely next word based on its training data and the patterns it has learned. It calculates probabilities using the context and the patterns it has observed in the training data.

Deep Learning and Model Complexity:

Large language models (LLMs) use a technique called deep learning, which involves multiple layers of interconnected neurons. These layers help the model understand language by capturing patterns in the text data. It's like learning step by step: starting with simple patterns and gradually understanding more complex ones. This allows LLMs to grasp the surrounding words and accordingly understand the meaning of each word so that the underlying meaning of sentence is maintained.

Fine-tuning and Transfer Learning:

After the initial training, LLMs can be further fine-tuned on specific tasks or domains.

Now, let's understand the meaning of fine-tuning first. Fine tuning is the process where we take a model that has been trained on large set of data and then we train it on our own custom data.

Let's understand the example, We taught a baby how to recognize the animals, then you are fine tuning this baby's knowledge for let's say identify " Dog".

Fine-tuning helps adapt the LLM to perform better on specific applications like question answering or text classification.

Now, transfer learning is the process where we use a pre-trained model and uses its knowledge to do a specific task. Let's take an example, as the baby knows how to identify dogs we can use the information learnt to identify various breeds of dog.

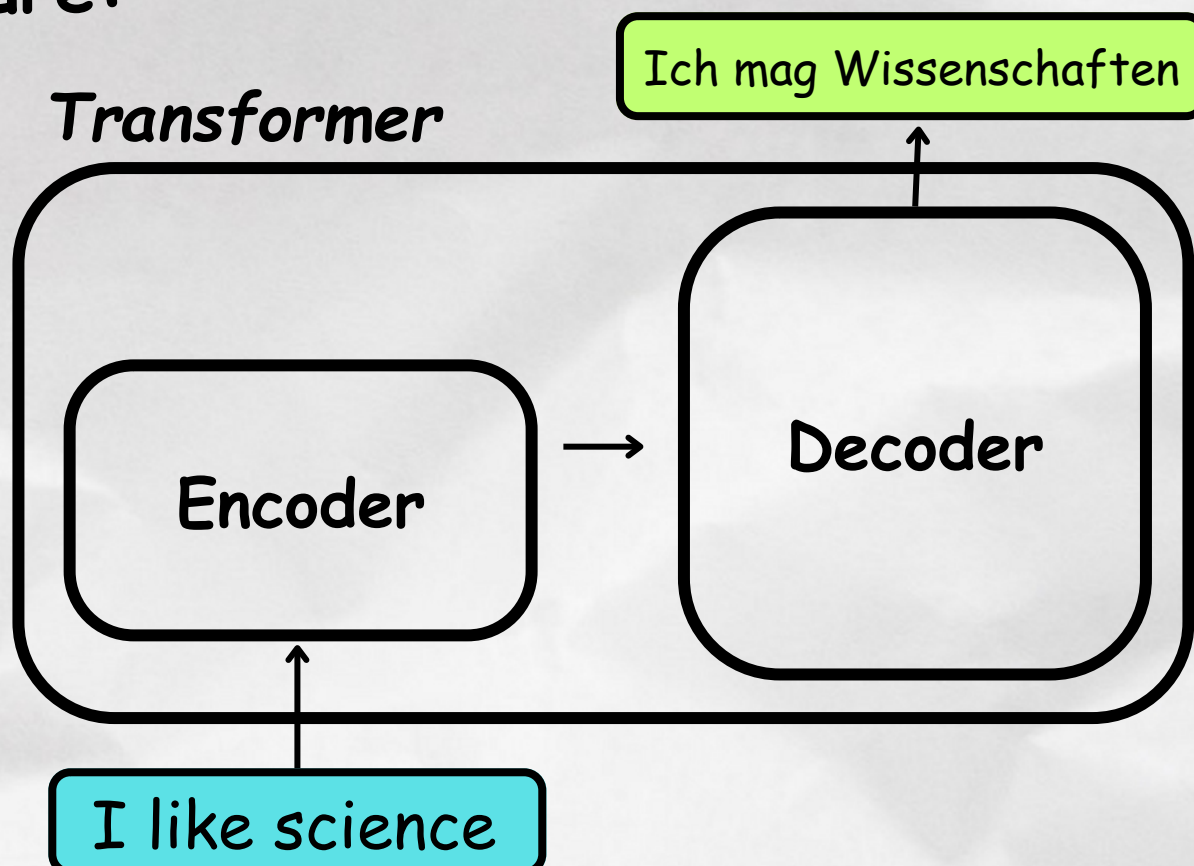
This involves training the model on a smaller dataset specific to the desired task.





Now, let's understand the architecture behind the basic understanding of the terms: (Point to be noted is here I will give the intuition only, the underlying mathematics behind it will be covered in detail in later articles!) Stay tuned!

It was introduced in the paper "Attention is all you need," published in December 2017. The following image shows a simplified version of the Transformer Architecture:-



The Transformer architecture consists of two fundamental components: the encoder and the decoder.

1. **Encoder**: The encoder's job is to understand the input text and figure out what it means. It takes in a bunch of words together and looks at them all at once. The important part of the encoder is something called self-attention. It helps the model focus on different words and understand how they relate to each other, which helps the model capture the meaning of the text.

So, think of the encoder as a super smart reader that looks at all the words together and uses self-attention to understand how they fit together and what they mean.



2. **Self-Attention**: Self-attention helps the model pay attention to different words in the input and understand how they are related. It calculates the importance of each word in relation to the others and adds them up to get a better understanding of meaning. This process helps the model capture how words are connected and understand the context of the text.

Think of self-attention as a way for the model to highlight important words and figure out how they all fit together. By doing this, the model can understand the relationships between words and get a better grasp of what the text is trying to say.

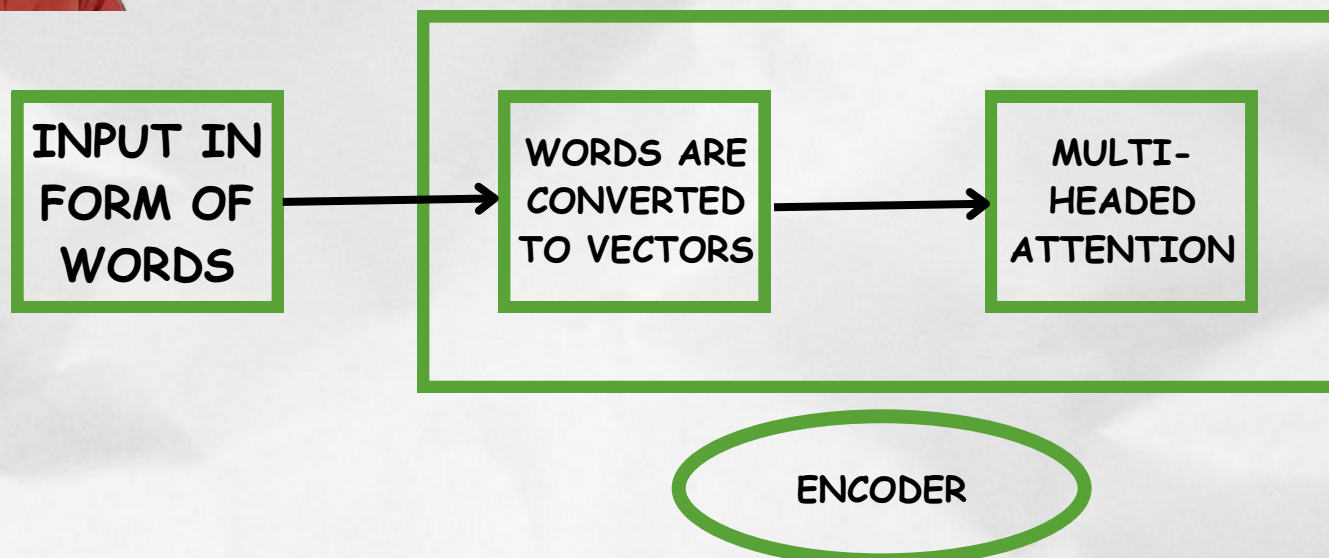


3. Multi-Head Attention:

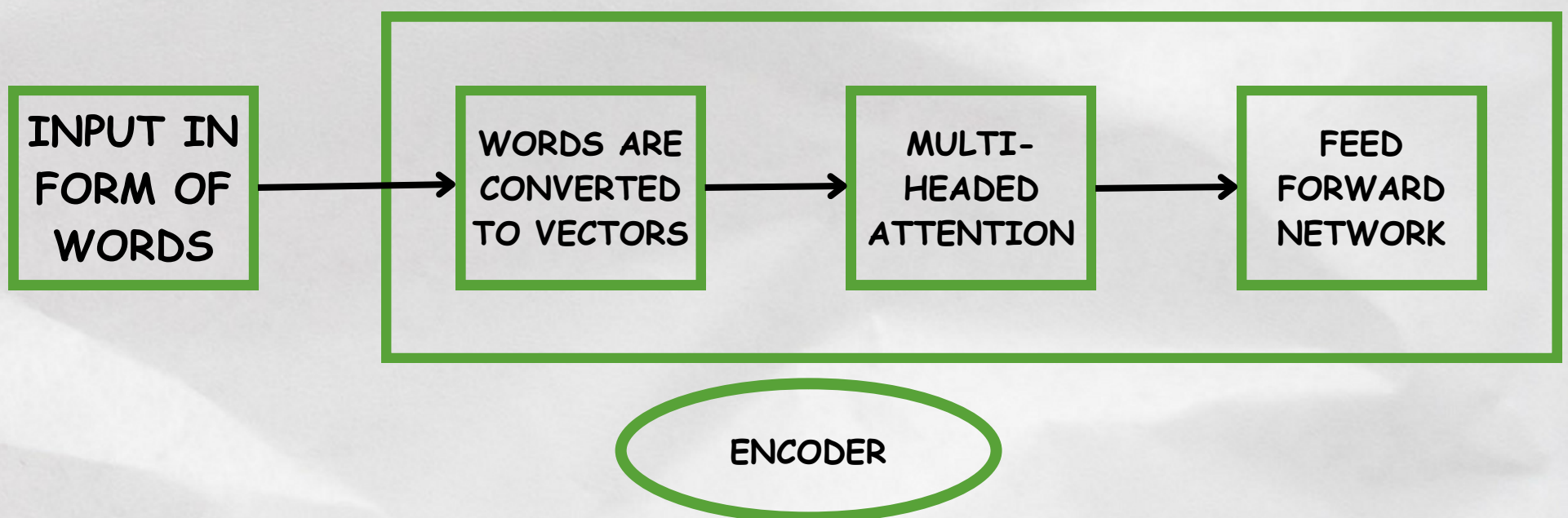
To make the model better understand different aspects of the text, we use multiple self-attention "heads." Each head pays attention to a different part of the text, like looking at different pieces of a puzzle. By doing this, the model can consider different ideas and patterns at the same time, which helps it understand the text from different perspectives.



Think of it like having multiple pairs of eyes looking at different parts of a picture. Each pair of eyes focuses on a specific area, and by combining what they see, we get a better overall understanding of the picture. Similarly, the multiple self-attention heads allow the model to see different parts of the text and put it all together to understand the whole story.

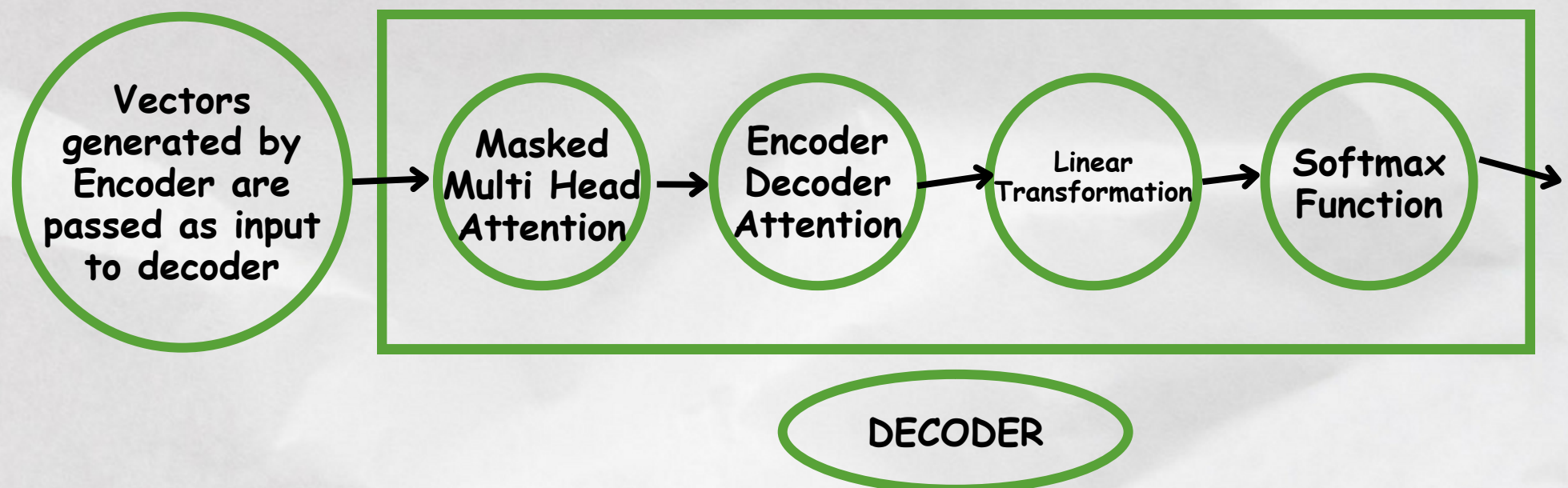


4. **Feed-Forward Networks**: After the model pays attention to different parts of the text and understands the relationships between words, it then uses a special type of neural network called a feed-forward network. This network looks at each word individually and makes it more powerful by adding extra information and capturing more complicated patterns in the text.



5. **Decoder**: A decoder takes the information that the encoder understood and uses it to generate the output text. It predicts the next word in the sequence by looking at both the input words and the words it has already generated.

Think of the decoder as a clever writer who uses what the encoder learned about the text to come up with the next word that fits well. It considers both the original words and the words it has already written to make sure everything flows smoothly and makes sense.



6. Self-Attention in the Decoder:

Just like the encoder, the decoder also pays attention to the words in the output it generates. It does this to understand how the words relate to each other. However, the decoder uses a special kind of attention called masked self-attention which helps the decoder stay focused on what it has already written and prevents it from peeking into the future words that it hasn't generated yet. It tries to hide the next word by using a [Mask].

Think of it like a game where the decoder is writing a story one word at a time. It pays attention to the words it has already written to make sure everything makes sense. But it's like having a rule that says you can't look ahead in the story to keep it coherent and logical. The masked self-attention helps the decoder follow this rule and create a story that flows smoothly and makes sense.



7. Encoder-Decoder Attention:

The decoder looks at the information the encoder understood and pays attention to the important parts. This helps the decoder align its generated output with the relevant information from the input.

Think of it like having a conversation - the decoder listens carefully to what the encoder understood and focuses on the important things the encoder said. This way, when the decoder responds, it can make sure its answer relates to the key points from the conversation.

8. Linear Transformation and Softmax:

The decoder takes its generated outputs and does two important things with them. First, it changes the outputs a bit using a special math operation called a linear transformation. Then, it uses another operation called softmax to assign probabilities to different words in its vocabulary.

This is done to ease choosing from the list of words by assigning them probability so that the word which is the best fit in the sentence and makes the correct sense is chosen.

The combination of the encoder and decoder in LLMs, along with their attention mechanisms, helps the model understand the meaning of the input text and generate output that makes sense. By training these models on big sets of data, they learn the patterns and rules of language, allowing them to create high-quality text.

The idea behind the transformer architecture is to use self-attention to understand how words relate to each other. This helps the model understand and generate language in a way that takes into account the context, making the output more meaningful and coherent.





Could you please consider supporting us
by sharing this post?

It would mean a lot to us. Give it a big
thumbs up and tag people who you think
will find this helpful. We appreciate your
support!

Content Strategist : Hanit Kaur
Graphic Designer : Adithya Prasad
Content Lead : Sumit Shukla

DON'T FORGET TO
SAVE 

Follow for more



**Session
With
Sumit**